

Foundational Reasoning and Feedback Protocol (FRFP): An Updated Formal Specification

The Lean-Corrected and Extended Account

Vijaya Nadendla^{1,†}

¹Independent Researcher

[†]Current address: 51 Linwood Dr, Monroe Township, NJ 08831

April 2026

Based on Lean 4.29.0 formal verification; 269 machine-checked theorems

Companion to: *FRFP Lean Formal Verification* (verification companion paper)

Abstract

The Foundational Reasoning and Feedback Protocol (FRFP) is a formal framework for structuring reasoning and evaluation in long-horizon human–AI workflows. It distinguishes between *explicit* processes, which operate on representations and are mechanically reproducible, and *tacit* processes, which determine correctness, validity, and acceptance. The framework enforces a one-way boundary between these domains, under which explicit computation produces artifacts that must be evaluated through tacit judgment.

This paper presents the FRFP theory in the form established by complete machine-checked formalization in Lean 4.

The formalization yields four refinements to the published theory:

1. **Basis reduction:** the original 11 premises reduce to 7 independent axioms. Four (ETS, RB, AE, MD) are derivable theorems, not independent assumptions.
2. **Precision corrections:** six definitions required strengthening — notably, credence bounds must be explicit, degradation requires clamping, and normal-form uniqueness holds only modulo navigation equivalence.
3. **New formal structures:** three types implicit in the original paper are made explicit — the `runToω` bijection, the `CorrectnessJudgment` type, and the lifted-action commutativity lemma.
4. **Novel theorems:** three non-trivial results emerge directly from proof obligations — a canonical first-unresolved-witness theorem, a governance-safe trace-extension theorem, and a protocol normal-form decomposition theorem.

This presentation treats the Lean formalization as authoritative: where the published paper and Lean code diverge, the Lean version is stated explicitly and the divergence is documented.

Contents

1	Introduction	5
2	The Kernel Category	7
2.1	Overview	7
2.2	Objects	7
2.3	Primitive Morphisms	7
2.4	Subcategories	8
2.5	Core Typing Theorems	8
2.6	Pipelines	9
2.7	Primitive Phase Partition	9
2.8	The No-Return Principle	9
2.9	Immutability of the Kernel	10
3	The Reduced Axiom Basis	11
3.1	From 11 Premises to 7	11
3.2	Derivable Theorems (No Longer Axioms)	11
3.3	Core Irreducible Premises	12
3.4	Interaction Irreducible Premises	12
3.5	Source Anchoring of External Assumptions	13
3.6	Decomposition: Skeleton vs Policy Residue	13
4	Reduction and Confluence	15
4.1	Overview	15
4.2	Task Decomposition Grammar	15
4.3	Reduction Relations	15
4.4	Normal Forms	16
4.5	Termination	16
4.6	Local Confluence	16
4.7	Global Confluence (Church–Rosser)	16
4.8	Normal Form Uniqueness (Corrected)	17
4.9	Semantic Correctness	17
4.10	Protocol Normal Form Theorem	17
5	Denotational and Operational Semantics	18
5.1	Overview	18
5.2	The Grothendieck Construction	18
5.3	Lean Correction: Reduction Acts on Objects	18
5.4	Semantic Functions	19
5.5	Runs and Trajectories	19
5.6	The runToω Bijection	19
5.7	Denotational Semantics of Pipelines	20
5.8	Navigation Layer	20
5.9	Immutability of Pipelines	21

6	Dynamic and Probabilistic Layer	22
6.1	Overview	22
6.2	Epistemic State	22
6.3	Degradation	22
6.4	Entropy Drift	23
6.5	Stopping Times	23
6.6	Five Proven Probability Theorems	23
6.7	Safe Horizon	24
6.8	Tacit Irreversibility	24
7	Collective and Governance Layer	25
7.1	Overview	25
7.2	Epistemic Algebra	25
7.3	Institutional Layer	25
7.4	Consensus Impossibility	26
7.5	Governance Framework	26
7.6	Step Compatibility	27
7.7	Governance Safe Extension Theorem	27
7.8	Entropy Drift in Collective Settings	27
7.9	Policy Residue in the Collective Layer	27
8	Novel Results from Formalization	28
8.1	Overview	28
8.2	Novel Theorem 1: First Unresolved Witness	28
8.3	Novel Theorem 2: Governance Safe Extension	28
8.4	Novel Theorem 3: Protocol Normal Form	29
8.5	Direct Consequences of the Reduced Basis	30
8.6	Novelty Classification	30
8.7	Open Questions	30

Notation

Symbol	Meaning
$\mathbf{C}_0$	The kernel category with objects $\{\emptyset, E_0, T_0\}$
E_0	The explicit object
T_0	The tacit object
$\mathbf{E}_{\text{cat}}$	The explicit subcategory (morphisms targeting E_0)
$\mathbf{T}_{\text{cat}}$	The tacit subcategory (morphisms targeting T_0)
Π	A pipeline (morphism in $\mathbf{C}_0$)
$\mathcal{P}$	A protocol trace (list of step records)
σ	A configuration (object in the Grothendieck construction)
$\equiv_{\nabla}$	Navigation equivalence relation
δ	Degradation operator on credence
$S(n)$	Survival function at step n
$h(n)$	Hazard rate at step n
ω	A stochastic trajectory
Ω	Sample space of stochastic trajectories

Throughout the paper, Lean theorems and definitions are referenced using the pattern `[Lean: module.theorem_name]`. All theorems marked `[Lean: \protect\global\let\T1\tellipsis.\kern\fontdimen3\font.\kern\fontdimen3\font.\kern\fontdimen3\font\T1\tellipsis]` have zero sorry and are accepted by the Lean 4.29.0 kernel.

1 Introduction

The Foundational Reasoning and Feedback Protocol (FRFP) is a formal framework for structuring reasoning and evaluation in systems that combine machine computation with human judgment. It is based on a fundamental distinction between *explicit* processes, which operate on representations and can be executed mechanically, and *tacit* processes, which determine correctness, validity, and acceptance.

Earlier presentations of FRFP introduced this structure as a conceptual and theoretical framework for long-horizon human–AI collaboration. This paper provides an updated and consolidated formal specification of the framework. The presentation reflects refinements arising from complete formalization, including simplification of the primitive basis, clarification of key definitions, and strengthening of structural constraints.

The purpose of this document is to present the current form of FRFP as a self-contained system. The focus is on precise definitions and derivable properties rather than on formal verification or proof mechanics. Where earlier versions differ, the refined formulation given here is taken as authoritative.

The remainder of the paper develops the framework starting from a minimal categorical kernel that captures the separation between explicit and tacit domains.

The FRFP Document Ecosystem

This paper is part of a broader research program organized into three layers. Understanding where this document fits will help the reader navigate the full body of work.

Theory layer. The mathematical foundations of FRFP are developed across three documents. *FRFPPaper Vol. 1* introduced the explicit–tacit separation, the representational boundary, and the formal protocol constraints. The present document supersedes that account with a consolidated and corrected specification, incorporating refinements that emerged from complete formalization. For readers who require a pedagogical introduction to the underlying mathematics, the *FRFP Companion Tutorial* provides accessible explanations of the key structures before any formal development. The *Lean Formalization* companion paper records the machine-checked verification of this theory in Lean 4, including 269 theorems, a basis reduction from 11 premises to 7, and novel results not present in earlier versions.

Engineering layer. *FRFPPaper Vol. 2* applies the theoretical framework to a corpus of canonical socio-technical case studies. The *FRFP Technical Specification* translates the abstract protocol constraints into implementable procedures for LLM-based reasoning pipelines. A series of *FRFP White Papers* (e.g., *Beyond Prompt Engineering*) provides engineering-facing interpretations of the protocol and its practical consequences.

Evaluation layer. *FRFP Vol. 3: Experimental Methods for Evaluating AI Agents* defines a general controlled-evaluation framework for AI reasoning systems. The *FRFP vs. Base Survey* applies that framework to empirically assess protocol-guided reasoning against baseline LLM outputs, using normalized one-page analytical summaries as the evaluation artifacts for blinded human raters.

Recommended reading path. A reader new to FRFP should begin with *FRFPPaper Vol. 1*, which provides the motivation, conceptual development, and theoretical arguments underlying the framework. The present document should then be read as the authoritative formal specification: it refines and consolidates the definitions from Vol. 1 but does not repeat its motivating discussion. Readers who find the mathematics unfamiliar should

consult the FRFP Companion Tutorial before or alongside this document. After this document, readers interested in machine-checked proofs should proceed to the Lean Formalization companion. Readers whose primary interest is deployment or empirical evaluation should continue to the Engineering and Evaluation layers in sequence.

Practitioners and engineers whose primary interest is implementation rather than formal theory may proceed directly to *FRFP Paper Vol. 2*, which provides an engineering-oriented exposition of the framework through detailed socio-technical case studies, and to the *FRFP Technical Specification* for operational protocol rules.

2 The Kernel Category

2.1 Overview

FRFP is built on a minimal three-object category $\mathbf{C}_0$ whose structure encodes the fundamental distinction between explicit (machine-manipulable) and tacit (human correctness-bearing) computation. This chapter defines $\mathbf{C}_0$, its six primitive morphisms, their typing rules, and the core theorems that follow directly from the definitions — without requiring any additional axioms.

2.2 Objects

Definition 2.1 (Kernel category objects). The kernel category $\mathbf{C}_0$ has exactly three objects:

$$\text{Ob}(\mathbf{C}_0) = \{\emptyset, E_0, T_0\}$$

Object	Name	Interpretation
$\emptyset$	Empty	The absent or uninitialized explicit state
E_0	Explicit object	Machine-manipulable representations
T_0	Tacit object	Human correctness-bearing knowledge

The object set is closed. No extensions to $\mathbf{C}_0$ are permitted. [**Lean:** `Frfp.Core.Kernel.Object`]

The objects are sometimes written with subscript notation: E_0 (the explicit *object*) is distinct from $\mathbf{E}_{\text{cat}}$ (the *category* of explicit morphisms). This distinction is enforced in the Lean formalization by separate types.

2.3 Primitive Morphisms

Definition 2.2 (Primitives). The six primitive morphisms of $\mathbf{C}_0$ are:

Primitive	Source	Target	Name	Role
RI	$\emptyset$	E_0	Representation Initiation	Creates an explicit representation
EC	E_0	E_0	Explicit Computation	AI processing within explicit space
ED	E_0	E_0	Explicit Diagnostics	AI checking/verification within explicit space
RB	E_0	T_0	Representational Backflow	The unique boundary crossing morphism
TE	T_0	T_0	Tacit Evaluation	Human evaluation within tacit space
HFD	T_0	T_0	Human Final Decision	Human closure within tacit space

[**Lean:** `Frfp.Core.Kernel.Primitive`]

The primitive set is a closed enumeration. No new primitives may be added without a complete re-audit of all downstream theorems.

2.4 Subcategories

Definition 2.3 (Explicit subcategory). $\mathbf{E}_{\text{cat}}$ is the full subcategory of morphisms targeting E_0 :

$$\mathbf{E}_{\text{cat}} = \{p : \text{Primitive} \mid p.\text{target} = E_0\} = \{\text{RI}, \text{EC}, \text{ED}\}$$

Definition 2.4 (Tacit subcategory). $\mathbf{T}_{\text{cat}}$ is the full subcategory of morphisms targeting T_0 :

$$\mathbf{T}_{\text{cat}} = \{p : \text{Primitive} \mid p.\text{target} = T_0\} = \{\text{RB}, \text{TE}, \text{HFD}\}$$

[Lean: Frfp.Core.Kernel.Ecat], [Lean: Frfp.Core.Kernel.Tcat]

The partition $\mathbf{E}_{\text{cat}} \cup \mathbf{T}_{\text{cat}}$ covers all six primitives and is disjoint.

2.5 Core Typing Theorems

The following theorems are proved directly from the primitive definitions — no additional axioms are required.

Theorem 2.5 (Explicit–Tacit Separation). *There is no primitive morphism from T_0 to E_0 :*

$$\forall p : \text{Primitive}, \neg(p.\text{source} = T_0 \wedge p.\text{target} = E_0)$$

Proof. By case analysis on the six primitives. None has source T_0 and target E_0 . $\square$ $\square$

[Lean: Frfp.Core.Kernel.no_morphism_T0_to_E0]

Remark 2.6. In the original published theory, ETS was listed as a primitive axiom. The Lean formalization shows it is a consequence of the primitive typing. See Chapter 3.

Theorem 2.7 (Uniqueness of the boundary morphism). *RB is the unique primitive crossing from E_0 to T_0 :*

$$\forall p : \text{Primitive}, (p.\text{source} = E_0 \wedge p.\text{target} = T_0) \Rightarrow p = \text{RB}$$

Proof. By case analysis on the six primitives. Only RB has source E_0 and target T_0 . $\square$ $\square$

[Lean: Frfp.Core.Kernel.RB_unique_boundary]

Theorem 2.8 (AI executability partition). *Define is_AI_executable as $\{\text{RI}, \text{EC}, \text{ED}\}$. Then:*

$$\forall p, \text{is_AI_executable}(p) \Rightarrow (p.\text{source} \in \{\emptyset, E_0\}) \wedge p.\text{target} = E_0$$

[Lean: Frfp.Core.Kernel.AI_preserves_explicit]

Corollary 2.9 (Human-exclusive tacit access). *RB, TE, and HFD are never AI-executable and always operate in or enter tacit space. [Lean: Frfp.Core.Kernel.is_human_only]*

2.6 Pipelines

Definition 2.10 (Pipeline). A pipeline Π is a morphism in $\mathbf{C}_0$: a triple $(\text{src}, \text{tgt}, p)$ where p is the underlying primitive.

Pipeline type	Source	Target	Underlying primitives
Explicit pipeline	$\emptyset$ or E_0	E_0	RI, EC, ED
Boundary pipeline	E_0	T_0	RB (unique)
Tacit pipeline	T_0	T_0	TE, HFD

Definition 2.11 (Pipeline composition). For pipelines $\Pi_1 : X \rightarrow Y$ and $\Pi_2 : Y \rightarrow Z$, their composition $\Pi_2 \circ \Pi_1 : X \rightarrow Z$ is defined when $\Pi_1.\text{target} = \Pi_2.\text{source}$. [**Lean:** `Frfp.Core.Kernel.pipeline_compose`]

Theorem 2.12 (Composition preserves typing). If $\Pi_1 : X \rightarrow Y$ and $\Pi_2 : Y \rightarrow Z$, then $(\Pi_2 \circ \Pi_1).\text{source} = X$ and $(\Pi_2 \circ \Pi_1).\text{target} = Z$. [**Lean:** `Frfp.Core.Kernel.pipeline_compose`]

2.7 Primitive Phase Partition

Definition 2.13 (Primitive phase). The phase of a primitive is defined by:

$$\text{phase}(p) = \begin{cases} \text{explicit} & \text{if } p \in \{\text{RI}, \text{EC}, \text{ED}\} \\ \text{boundary} & \text{if } p = \text{RB} \\ \text{tacit} & \text{if } p \in \{\text{TE}, \text{HFD}\} \end{cases}$$

[**Lean:** `Frfp.Minimal.BasisSweep.phaseOf`]

Theorem 2.14 (Phase partition). The phase function is total: every primitive has exactly one phase. [**Lean:** `Frfp.Minimal.BasisSweep.phase_partition`]

Theorem 2.15 (Boundary uniqueness by phase). A primitive has boundary phase if and only if it is RB. [**Lean:** `Frfp.Minimal.BasisSweep.phase_boundary_iff`]

2.8 The No-Return Principle

Theorem 2.16 (No tacit-to-explicit return). There is no sequence of primitives that goes from T_0 to E_0 :

$$\nexists p_1, \dots, p_n : \text{Primitive}, \quad p_1.\text{source} = T_0 \wedge p_n.\text{target} = E_0$$

This follows from Theorem 1.1 (ETS) by induction: every primitive either stays in tacit space ($T_0 \rightarrow T_0$) or goes to E_0 from $\emptyset$ or E_0 .

The one-directional boundary $E_0 \xrightarrow{\text{RB}} T_0$ is a genuine asymmetry in the categorical structure: not a functor, not an equivalence, but a one-way transition that formalizes the tacit grounding of AI outputs by human judgment.

2.9 Immutability of the Kernel

Note 2.17 (Kernel immutability). The definitions of `Object`, `Primitive`, `Primitive.source`, and `Primitive.target` are immutable. No module in the library may shadow, redefine, or extend them.

This is enforced architecturally: `Kernel.lean` has no imports other than Lean’s standard library, and all other modules depend on it. Any change to Kernel invalidates all 3305 downstream build jobs. [**Lean:** `Frfp.Core.Kernel---noextends,noshadows`]

3 The Reduced Axiom Basis

3.1 From 11 Premises to 7

The original published FRFP theory enumerates 11 named premises: ETS, RB, AE, MD, HEG, HEC, CP, IL, AR, NTER, CSC. The Lean formalization establishes that four of these (ETS, RB, AE, MD) are theorems derivable directly from the kernel definitions. The minimal independent basis therefore has exactly **7 primitive assumptions**.

Premise	Original status	Lean status	Location
ETS	Axiom	Theorem (kernel typing)	Thm 2.1
RB	Axiom	Theorem (kernel enumeration)	Thm 2.2
AE	Axiom	Theorem (AI-executable typing)	Thm 2.3
MD	Axiom	Theorem (AI/human partition)	Thm 2.4
HEG	Axiom	Axiom (retained)	§3.3
HEC	Axiom	Axiom (retained)	§3.3
CP	Axiom	Axiom (retained)	§3.3
IL	Axiom	Axiom (retained)	§3.4
AR	Axiom	Axiom (retained)	§3.4
NTER	Axiom	Axiom (retained)	§3.4
CSC	Axiom	Axiom (retained)	§3.4

3.2 Derivable Theorems (No Longer Axioms)

ETS — Explicit–Tacit Separation

Theorem 3.1 (ETS is a theorem). *There is no primitive from T_0 to E_0 :*

$$\forall p : \text{Primitive}, \neg(p.\text{source} = T_0 \wedge p.\text{target} = E_0)$$

[Lean: Frfp.Minimal.ReducedBasis.ETS_derivable] — proved by delegating to no_morphism_T0_to_E0.

Note 3.2. ETS holds because the primitive enumeration simply contains no $T_0 \rightarrow E_0$ morphism. It is a fact about the closed enumeration, not an independent assumption.

RB — Boundary Uniqueness

Theorem 3.3 (RB uniqueness is a theorem). *RB is the unique primitive from E_0 to T_0 :*

$$\forall p : \text{Primitive}, (p.\text{source} = E_0 \wedge p.\text{target} = T_0) \Rightarrow p = \text{RB}$$

[Lean: Frfp.Minimal.ReducedBasis.RB_derivable]

AE — AI-Explicit Restriction

Theorem 3.4 (AE is a theorem). *Every AI-executable primitive stays within explicit space:*

$$\forall p, \text{is_AI_executable}(p) \Rightarrow (p.\text{source} \in \{\emptyset, E_0\}) \wedge p.\text{target} = E_0$$

[Lean: Frfp.Minimal.ReducedBasis.AE_derivable]

MD — Mode Discipline

Theorem 3.5 (MD is a theorem). *AI-executable primitives always target E_0 ; human-only primitives always target T_0 :*

$$(\forall p, \text{is_AI_executable}(p) \Rightarrow p.\text{target} = E_0) \wedge (\forall p, \text{is_human_only}(p) \Rightarrow p.\text{target} = T_0)$$

[Lean: Frfp.Minimal.ReducedBasis.MD_derivable]

3.3 Core Irreducible Premises

The three core premises below are retained as genuine axioms. They capture domain commitments that cannot be derived from the kernel structure alone.

HEG — Human-Exclusive Grounding

Axiom 3.6 (Human-Exclusive Grounding (HEG)). Tacit evaluation and human final decision — the primitives TE and HFD — are exclusively executable by humans. No AI system can perform tacit grounding.

Formal skeleton: $\text{HEG}_{\text{skel}} : \text{TE.target} = T_0 \wedge \text{HFD.target} = T_0$

[Lean: Frfp.Minimal.ReducedBasis.HEG_skeleton_derivable]

Policy residue: The normative claim — that only humans may execute TE and HFD — is a governance premise.

HEC — Human-Exclusive Closure

Axiom 3.7 (Human-Exclusive Closure (HEC)). The final-decision primitive HFD is an endomorphism of tacit space that only humans may apply.

Formal skeleton: $\text{HEC}_{\text{skel}} : \text{HFD.source} = T_0 \wedge \text{HFD.target} = T_0$

[Lean: Frfp.Minimal.ReducedBasis.HEC_skeleton_derivable]

CP — Compositional Pipelines

Axiom 3.8 (Compositional Pipelines (CP)). Pipelines compose in a way that preserves the source and target boundary:

$$(\Pi_2 \circ \Pi_1).\text{source} = \Pi_1.\text{source}, \quad (\Pi_2 \circ \Pi_1).\text{target} = \Pi_2.\text{target}$$

[Lean: Frfp.Minimal.ReducedBasis.CP_skeleton_derivable]

3.4 Interaction Irreducible Premises

IL — Intent Locking

Axiom 3.9 (Intent Locking (IL)). All AI steps in a protocol trace share a common intent tag. An AI step cannot unilaterally change the declared intent of the interaction:

$$\forall a, b \in \mathcal{P}, a.\text{isAI} = \top \wedge b.\text{isAI} = \top \Rightarrow a.\text{intentTag} = b.\text{intentTag}$$

Source anchor: Temporal safety / invariant-over-traces semantics. [Pnueli 1977]

AR — Ambiguity Resolution

Axiom 3.10 (Ambiguity Resolution (AR)). Every ambiguous step in a protocol trace must be clarified before the interaction proceeds:

$$\forall a \in \mathcal{P}, a.\text{ambiguous} = \top \Rightarrow a.\text{clarified} = \top$$

Source anchor: Liveness over traces; safety/liveness decomposition. [Alpern–Schneider 1985]

NTER — No Tacit Emulation/Reconstruction

Axiom 3.11 (No Tacit Emulation or Reconstruction (NTER)). No AI step attempts to construct a morphism from T_0 to E_0 , i.e., no AI step attempts to emulate or substitute for tacit human judgment.

Formal skeleton: $\forall p, p.\text{source} = T_0 \Rightarrow p.\text{target} \neq E_0$

[Lean: Frfp.Minimal.ReducedBasis.NTER_skeleton_derivable]

CSC — Correctness Stability Constraint

Axiom 3.12 (Correctness Stability Constraint (CSC)). Once a step in a protocol trace is correctness-locked, all subsequent steps are correctness-locked. Correctness locking is monotone:

$$(\exists a \in \mathcal{P}, a.\text{correctnessLocked} = \top) \Rightarrow \forall b \in \mathcal{P}, b.\text{correctnessLocked} = \top$$

Source anchor: Temporal safety (invariant preservation). [Pnueli 1977]

3.5 Source Anchoring of External Assumptions

Modules beyond the kernel require additional axioms from established mathematics.

Family	Representative axioms	Primary source
IEEE 754 float arithmetic	Associativity, commutativity, order	[IEEE 754-2019]
Probability / measure	Countable additivity, normalization	[Billingsley 1995]
Stopping times	Measurability, Markov property	[Billingsley 1995] §36
Stochastic tail bounds	Markov inequality, geometric decay	[Durrett 2019]
Term rewriting / ARS	Termination, confluence	[Baader–Nipkow 1998]
Newman’s Lemma	Confluence from termination	[Newman 1942]
Category / groupoid	Associativity, identity, inverses	[Mac Lane 1971]
Real analysis	Geometric limit-to-zero	[Rudin 1976]
Temporal safety	Safety properties over traces	[Pnueli 1977]
Safety vs liveness	Decomposition of liveness	[Alpern–Schneider 1985]

[Lean: Frfp.Minimal.ReducedBasis.sourceBackedFamilies]

3.6 Decomposition: Skeleton vs Policy Residue

Each of the 7 retained axioms decomposes into two parts:

- **Skeleton:** the mathematically structural part, derivable or source-anchored to published mathematics.
- **Policy residue:** the irreducible normative/governance commitment.

Axiom	Skeleton	Policy residue
HEG	TE, HFD target T_0 (Lean-proved)	Only humans may execute these
HEC	HFD is a T_0 -endomorphism (Lean-proved)	No AI substitution for closure
CP	Composition preserves source/-target (Lean-proved)	Pipelines are the only valid composition
IL	Trace invariant (Pnueli 1977)	Intent is a shared, locked resource
AR	Liveness (Alpern–Schneider 1985)	Ambiguity must be resolved
NTer	Structural = ETS (Lean-proved)	AI prohibited from tacit emulation attempts
CSC	Safety/monotonicity (Pnueli 1977)	Correctness lock is irreversible

The policy residue of all seven axioms is where the human-authority commitments of FRFP live. These cannot be formalized as mathematical theorems — they are governance premises.

[**Lean:** `Frfp.Minimal.ReducedBasis.DecompositionStatus`]

4 Reduction and Confluence

4.1 Overview

This chapter defines the syntactic layer of FRFP — the Task Decomposition Grammar (TDG) that generates pipeline terms — and the semantic reduction rules that normalize them. We prove two structural properties: *termination* (every reduction sequence is finite) and *confluence* (the order of reductions does not affect the final result). Together these establish that every pipeline term has a unique normal form modulo navigation equivalence.

4.2 Task Decomposition Grammar

Definition 4.1 (TDG terms). A TDG term over the kernel primitives is a well-typed finite sequence:

$$t ::= \varepsilon \mid p \cdot t$$

where p is a primitive and t is a TDG term, subject to the typing constraint that consecutive primitives compose.

TDG terms are syntactic objects. A TDG term *denotes* a pipeline (morphism) in $\mathbf{C}_0$ when interpreted by the semantic function $\llbracket \cdot \rrbracket$ (Chapter 5).

Definition 4.2 (Free category). The free category $\mathbf{F}$ generated by the six primitives has:

- Objects: $\{\emptyset, E_0, T_0\}$
- Morphisms: all well-typed finite compositions of primitives

$\mathbf{F}$ is the syntactic model; $\mathbf{C}_0$ is its semantic quotient.

4.3 Reduction Relations

Definition 4.3 (Explicit reduction). A reduction relation $\rightarrow_e$ on TDG terms captures explicit-space simplifications:

$$EC \cdot EC \rightarrow_e EC, \quad ED \cdot EC \rightarrow_e ED$$

together with similar rules for all combinations of $\{\text{RI}, \text{EC}, \text{ED}\}$.

Definition 4.4 (Tacit reduction). A reduction relation $\rightarrow_t$ on TDG terms captures tacit-space simplifications:

$$TE \cdot TE \rightarrow_t TE, \quad \text{HFD} \cdot TE \rightarrow_t \text{HFD}$$

Definition 4.5 (Combined reduction). The combined reduction $\rightarrow_c = \rightarrow_e \cup \rightarrow_t$, with contextual closure. The reflexive-transitive closure is written $\rightarrow_c^*$. `[Lean: Frfp.Core.OperationalSemantics]`, `[Lean: Frfp.Core.SemanticCorrectness]`

4.4 Normal Forms

Definition 4.6 (Normal form). A TDG term t is in *normal form* if no reduction rule applies. A normal form has the structure:

$$t_{\text{nf}} = \underbrace{e_1 \cdots e_k}_{\text{explicit block}} \cdot \underbrace{\text{RB}^?}_{\text{optional boundary}} \cdot \underbrace{h_1 \cdots h_m}_{\text{tacit block}}$$

where each $e_i \in \{\text{RI}, \text{EC}, \text{ED}\}$, $\text{RB}^?$ is either RB or absent, and each $h_j \in \{\text{TE}, \text{HFD}\}$.

[Lean: Frfp.Minimal.BasisSweep.PrimitiveTraceAdmissibleStrong]

This three-block structure is the **protocol normal form**. It reflects the operational semantics of an FRFP interaction: explicit work first, then (optionally) handoff to human tacit evaluation.

4.5 Termination

Theorem 4.7 (Termination). *Every reduction sequence in $\rightarrow_c$ is finite.* [Lean: Frfp.Core.ConfluenceProof---terminationcomponent]

Proof. Define a measure $\mu(t)$ as the number of consecutive same-phase pairs in t . Each reduction step strictly decreases $\mu(t)$. Since $\mu(t) \geq 0$, all sequences terminate. $\square$ $\square$

4.6 Local Confluence

Definition 4.8 (Local confluence). The relation $\rightarrow_c$ is *locally confluent* if for any t, t_1, t_2 with $t \rightarrow_c t_1$ and $t \rightarrow_c t_2$, there exists t_3 with $t_1 \rightarrow_c t_3$ and $t_2 \rightarrow_c t_3$.

Theorem 4.9 (Local confluence of $\rightarrow_c$). *The combined reduction is locally confluent.* [Lean: Frfp.Core.ConfluenceProof.diamond_property]

Proof. By case analysis on pairs of one-step reductions from t . Since explicit and tacit reductions act on disjoint parts of the term (by the ETS theorem), any two one-step reductions are either in the same phase at non-overlapping positions (trivially confluent), in the same phase at overlapping positions (resolved by the phase-specific confluence lemma), or in different phases (independently applicable in either order). $\square$ $\square$

Note 4.10. Lifted action commutativity — navigation actions commute with explicit reduction — is axiomatized from the operational semantics. [Lean: Frfp.Core.Confluence.lifted_action_commutates]

This property is implicit in the original published theory. The Lean formalization makes it an explicit required lemma.

4.7 Global Confluence (Church–Rosser)

Theorem 4.11 (Confluence). *By Newman’s Lemma, termination together with local confluence implies global confluence:*

$$\forall t, t_1, t_2, \quad t \rightarrow_c t_1 \wedge t \rightarrow_c t_2 \Rightarrow \exists t_3, \quad t_1 \rightarrow_c t_3 \wedge t_2 \rightarrow_c t_3$$

[Lean: Frfp.Core.ConfluenceProof.confluence]

Source: *Newman’s Lemma* [Newman 1942]; abstract reduction systems [Baader–Nipkow 1998] Ch. 2.

4.8 Normal Form Uniqueness (Corrected)

Theorem 4.12 (Normal form uniqueness modulo navigation equivalence). *For any TDG term t , all normal forms reachable from t are navigation-equivalent:*

$$\forall t, \forall \text{nf}_1, \text{nf}_2, \quad t \rightarrow_c \text{nf}_1 \wedge t \rightarrow_c \text{nf}_2 \wedge \text{IsNF}(\text{nf}_1) \wedge \text{IsNF}(\text{nf}_2) \Rightarrow \text{nf}_1 \equiv_{\nabla} \text{nf}_2$$

[Lean: Frfp.Core.SemanticCorrectness.unique_nf_mod_nav]

Lean Correction

The original published theory states normal forms are *literally unique* (equal). The Lean formalization shows they are unique only *modulo navigation equivalence* $\equiv_{\nabla}$. Two normal forms from the same term may differ in navigation annotations while denoting the same pipeline.

Definition 4.13 (Navigation equivalence). Two terms $t_1 \equiv_{\nabla} t_2$ if they differ only in navigation morphisms (admissible path annotations added by the navigation layer). Navigation morphisms do not change the computational content of a term. [Lean: Frfp.Core.SemanticCorrectness.NavEquivalent]

4.9 Semantic Correctness

Theorem 4.14 (Observable invariance). *For any term t and its normal form nf , the observable correctness judgment is preserved:*

$$\text{obs}(t) = \text{obs}(\text{nf})$$

where $\text{obs} = \gamma \circ \text{Sem} \circ \text{initial}$ is the composition of semantic evaluation, correctness judgment extraction, and trajectory initialization. [Lean: Frfp.Core.SemanticCorrectness.obsCorrect_invariant]

Definition 4.15 (Correctness judgment type). The correctness judgment type has three values:

$$\text{CorrectnessJudgment} = \{\text{correct}, \text{incorrect}, \text{unknown}\}$$

This is a new type made explicit by the Lean formalization; it was implicit in the original paper. [Lean: Frfp.Core.TacitDependence.CorrectnessJudgment]

4.10 Protocol Normal Form Theorem

Theorem 4.16 (Protocol normal form). *Any strongly admissible primitive trace decomposes as an explicit block, optional boundary crossing, and tacit block:*

$$\forall \mathcal{T}, \text{AdmissibleStrong}(\mathcal{T}) \Rightarrow \exists e_1 \dots e_k, h_1 \dots h_m, b \in \{\top, \perp\},$$

$$\mathcal{T} = [e_1, \dots, e_k] ++ (\text{if } b \text{ then } [\text{RB}] \text{ else } []) ++ [h_1, \dots, h_m]$$

[Lean: Frfp.Minimal.BasisSweep.protocol_normal_form_theorem] — Machine-checked proof.

5 Denotational and Operational Semantics

5.1 Overview

This chapter defines the semantic interpretation of FRFP terms. Two complementary views are presented:

- **Denotational semantics:** TDG terms denote morphisms in a Grothendieck construction over indexed categories.
- **Operational semantics:** pipeline execution is modeled as a relation on runs, connected to a stochastic trajectory space via the `runToω` bijection.

Both views contain significant structures made explicit by the Lean formalization that were implicit or absent in the original published theory.

5.2 The Grothendieck Construction

FRFP uses a Grothendieck construction to model configurations — the full state of a pipeline execution including both categorical structure and index data.

Definition 5.1 (Indexed category). Define a contravariant functor:

$$\mathbf{E}_{\text{idx}} : \mathbb{N}^{\text{op}} \rightarrow \mathbf{Cat}$$

mapping natural number indices n to categories $\mathbf{E}_{\text{idx}}(n)$ and index morphisms (decrements) to functors between them.

Definition 5.2 (Grothendieck object / configuration). A configuration σ is an object in the Grothendieck construction $\int \mathbf{E}_{\text{idx}}$:

$$\sigma = (n, X_n) \quad \text{where } n \in \mathbb{N}, X_n \in \text{Ob}(\mathbf{E}_{\text{idx}}(n))$$

[Lean: `Frfp.Core.Grothendieck.GrothendieckObject`]

Definition 5.3 (Grothendieck morphism). A morphism between configurations $(n, X_n) \rightarrow (m, X_m)$ consists of an index morphism $n \rightarrow m$ and a morphism in the base category compatible with the reindexing functor. [Lean: `Frfp.Core.Grothendieck.GrothendieckMorphism`]

5.3 Lean Correction: Reduction Acts on Objects

Lean Correction P-01

The original published theory is ambiguous about whether reduction steps operate on configurations, morphisms, or syntactic terms. The Lean formalization resolves this: **reduction is a relation on configurations** (Grothendieck objects), not on morphisms.

Definition 5.4 (Reduction step). A reduction step is a relation $\rightarrow_r \subseteq \text{GrothendieckObject} \times \text{GrothendieckObject}$.

Theorem 5.5 (Reduction induces morphism). *If $\sigma_1 \rightarrow_r \sigma_2$, then there exists a Grothendieck morphism m with $m.\text{source} = \sigma_1$ and $m.\text{target} = \sigma_2$:*

$$\sigma_1 \rightarrow_r \sigma_2 \Rightarrow \exists m : \text{GrothendieckMorphism}, m.\text{src} = \sigma_1 \wedge m.\text{tgt} = \sigma_2$$

[Lean: Frfp.Core.Grothendieck.reduction_induces_morphism]

5.4 Semantic Functions

Definition 5.6 (Semantic evaluation). The function Sem maps a configuration to an object in the kernel category:

$$\text{Sem} : \text{GrothendieckObject} \rightarrow \text{Object}$$

[Lean: Frfp.Core.TacitDependence.Sem]

Definition 5.7 (Correctness evaluation). The function γ maps a kernel object to a correctness judgment:

$$\gamma : \text{Object} \rightarrow \text{CorrectnessJudgment}$$

[Lean: Frfp.Core.TacitDependence.gamma]

Definition 5.8 (Observable correctness). The observable correctness function obs over runs:

$$\text{obs} = \gamma \circ \text{Sem} \circ \text{initial} : \text{Runs}(P) \rightarrow \text{CorrectnessJudgment}$$

[Lean: Frfp.Core.TacitDependence.obs]

Note 5.9. Sem , γ , obs , and $\text{CorrectnessJudgment}$ are all new explicit types introduced by the Lean formalization. They were implicit in the original paper’s treatment of tacit-dependent correctness.

5.5 Runs and Trajectories

Definition 5.10 (Run). A run ρ of program P is a maximal execution trace. The type of runs of P is written $\text{Runs}(P)$.

Definition 5.11 (Allowed stochastic trajectory). The sample space Ω is the full space of stochastic trajectories. The *allowed* subspace $\Omega_{\text{allowed}} \subseteq \Omega$ contains exactly those trajectories consistent with a valid run of P .

5.6 The runToω Bijection

Lean Correction P-02

Remark A.104 in the original paper asserts a correspondence between runs and allowed stochastic trajectories, but does not define it explicitly. The Lean formalization requires a concrete injective map with a round-trip property.

Definition 5.12 ($\text{runTo}\omega$). The run-to-trajectory correspondence:

$$\text{runTo}\omega : \{\rho \mid \text{Runs}(P) \rho\} \rightarrow \Omega_{\text{allowed}}$$

with companion inverse:

$$\omega\text{ToRun} : \Omega_{\text{allowed}} \rightarrow \{\rho \mid \text{Runs}(P) \rho\}$$

[Lean: Frfp.Core.OperationalSemantics.runToOmega], [Lean: Frfp.Core.OperationalSemantics.omegaToRun]

Axiom 5.13 ($\text{runTo}\omega$ injectivity).

$$\text{runTo}\omega(\rho_1) = \text{runTo}\omega(\rho_2) \Rightarrow \rho_1 = \rho_2$$

[Lean: Frfp.Core.OperationalSemantics.runToOmega_injective]

Axiom 5.14 ($\text{runTo}\omega$ round-trip).

$$\omega\text{ToRun}(\text{runTo}\omega(\rho)) = \rho$$

[Lean: Frfp.Core.OperationalSemantics.runToOmega_roundtrip]

Note 5.15. Injectivity and the round-trip property are axiomatized. A complete proof would require formalizing the probability space construction. This is an open proof obligation.

Interpretation: $\text{runTo}\omega$ is the formal bridge between the operational semantics (discrete execution traces) and the probabilistic semantics (measure on trajectory space). Without this bridge, probabilistic properties of FRFP cannot be stated as theorems about runs.

5.7 Denotational Semantics of Pipelines

Definition 5.16 (Pipeline denotation). The denotational semantics $\llbracket \cdot \rrbracket$ maps a TDG term to its morphism in $\mathbf{C}_0$:

$$\llbracket \varepsilon \rrbracket = \text{id}, \quad \llbracket p \cdot t \rrbracket = \llbracket p \rrbracket \circ \llbracket t \rrbracket$$

Theorem 5.17 (Denotational–operational agreement). *For any TDG term t and its reduct t' , the denotations are equal:*

$$t \rightarrow_c t' \Rightarrow \llbracket t \rrbracket = \llbracket t' \rrbracket$$

[Lean: Frfp.Core.Semantics]

5.8 Navigation Layer

Definition 5.18 (Navigation morphism). A navigation morphism is an admissible path annotation on a TDG term, recording which reachable states were visited during reduction. Navigation morphisms form a groupoid.

Definition 5.19 (Navigation equivalence). Two terms $t_1 \equiv_{\nabla} t_2$ if they are equal modulo navigation morphism annotations.

Theorem 5.20 (Navigation equivalence is an equivalence relation). • *Reflexivity:*

$$t \equiv_{\nabla} t.$$

$$\bullet \text{ Symmetry: } t_1 \equiv_{\nabla} t_2 \Rightarrow t_2 \equiv_{\nabla} t_1.$$

$$\bullet \text{ Transitivity: } t_1 \equiv_{\nabla} t_2 \wedge t_2 \equiv_{\nabla} t_3 \Rightarrow t_1 \equiv_{\nabla} t_3.$$

[Lean: Frfp.Core.SemanticCorrectness.nav_equivalence_is_equivalence]

5.9 Immutability of Pipelines

Theorem 5.21 (Pipeline immutability). *Executing a pipeline does not modify its structure. A pipeline’s source, target, and primitive are invariant under execution:*

$$\text{execute}(\Pi).\text{structure} = \Pi.\text{structure}$$

[Lean: Frfp.Core.ImmutabilityProof]

This theorem captures a key FRFP invariant: pipelines are static specifications. Execution produces outputs but does not rewrite the pipeline specification itself.

6 Dynamic and Probabilistic Layer

6.1 Overview

This chapter covers two connected aspects of FRFP dynamics:

1. **Epistemic state and credence degradation** — how tacit knowledge degrades over time when not refreshed by human judgment.
2. **Probabilistic stopping times and survival analysis** — the stochastic model of pipeline resolution, connecting to the runToω bijection.

6.2 Epistemic State

Definition 6.1 (Tacit state). A tacit state records the current quality of human-grounded knowledge in a pipeline execution. It is represented as a record with fields including a credence value and a degradation parameter.

Definition 6.2 (Credence). The credence of a tacit state is a rational number:

$$c \in [0, 1] \subset \mathbb{Q}$$

Lean Correction P-07

The original paper uses credence notation without stating bounds explicitly. The Lean formalization enforces bounds by construction:

```
structure Credence where
  value   : Rat
  nonneg  : 0 ≤ value
  le_one  : value ≤ 1
```

[Lean: Frfp.Core.InstitutionalLayer.Credence]

Without these structural bounds, any arithmetic on credence could produce values outside $[0, 1]$, making the tacit degradation theorem unprovable.

6.3 Degradation

Definition 6.3 (Degradation operator δ). The degradation operator models the decay of tacit correctness over time when not refreshed by human evaluation:

$$\delta : \text{TacitState} \rightarrow \text{TacitState}$$

Definition 6.4 (Degradation formula — corrected). Given credence $c \in [0, 1]$ and degradation rate $d \geq 0$, after t steps the credence is:

$$c'(t) = \max\left(0, \min\left(1, c - d \cdot t\right)\right)$$

Lean Correction P-07 (cont.)

The original paper writes $c'(t) = c - d \cdot t$ without clamping. The Lean formalization requires explicit clamping to keep credence within $[0, 1]$; without it, the non-negativity proof obligation fails.

Theorem 6.5 (Degradation monotonicity). *Credence is monotonically non-increasing under repeated degradation:*

$$t_1 \leq t_2 \Rightarrow c'(t_2) \leq c'(t_1)$$

[Lean: Frfp.Core.DynamicLayer.degradation_monotone]

6.4 Entropy Drift

Definition 6.6 (Entropy drift). The entropy of the tacit component of a pipeline increases monotonically when no human evaluation step occurs between AI steps:

$$\Delta H = H(t_2) - H(t_1) \geq 0 \quad \text{for } t_1 \leq t_2 \text{ with no HFD/TE between them}$$

Theorem 6.7 (Entropy drift monotonicity). *Under continuous AI-only execution (no TE or HFD steps), the entropy of the tacit component is non-decreasing.* [Lean: Frfp.Core.DynamicLayer.entropy_drift_monotone]

Entropy drift formalizes the FRFP claim that purely AI-driven pipelines without human checkpoints progressively lose tacit validity. The TE primitive resets the drift by incorporating human evaluation.

6.5 Stopping Times

Definition 6.8 (Stopping time). A stopping time τ for pipeline P is a measurable random variable:

$$\tau : \Omega \rightarrow \mathbb{N} \cup \{\infty\}$$

such that for all $n \in \mathbb{N}$, the event $\{\tau \leq n\}$ is measurable with respect to the filtration generated by the first n steps of the run.

[Lean: Frfp.Core.Probability.StoppingTime]

Definition 6.9 (Hazard rate). The hazard rate $h(n)$ is the conditional probability that the pipeline resolves at step n , given it has not resolved before step n :

$$h(n) = \mathbb{P}(\tau = n \mid \tau \geq n) = \frac{S(n-1) - S(n)}{S(n-1)}$$

Definition 6.10 (Survival function). The survival function $S(n)$ is the probability that the pipeline has not yet resolved by step n :

$$S(n) = \mathbb{P}(\tau > n)$$

6.6 Five Proven Probability Theorems

The following five theorems were axiomatized in an earlier version of the formalization (March 2026) and subsequently proved using Lean’s Mathlib library (April 2026). They are now machine-checked theorems, not assumptions.

Theorem 6.11 (Survival product formula).

$$S(N) = \prod_{k=0}^N (1 - h(k))$$

[Lean: Frfp.Core.ProbabilityProven.survival_product_formula]

Proof. By induction on N , using the definition of conditional probability and the product rule for independent events. $\square$ $\square$

Theorem 6.12 (Hazard–survival relation).

$$h(n) = \frac{S(n-1) - S(n)}{S(n-1)}$$

[Lean: Frfp. Core. ProbabilityProven. hazard_survival_relation]

Theorem 6.13 (Survival monotonicity).

$$N \leq M \Rightarrow S(M) \leq S(N)$$

[Lean: Frfp. Core. ProbabilityProven. survival_monotone]

Theorem 6.14 (Geometric survival). *When the hazard rate is constant, $h(n) = h$ for all n :*

$$S(n) = (1 - h)^{n+1}$$

[Lean: Frfp. Core. ProbabilityProven. geometric_survival]

Theorem 6.15 (Infinite product limit). *When $h(k) > 0$ for all k :*

$$\prod_{k=0}^{\infty} (1 - h(k)) = 0$$

[Lean: Frfp. Core. ProbabilityProven. infinite_product_limit]

Note 6.16. The transition of these five theorems from axioms to machine-checked results demonstrates that the FRFP probabilistic foundations are sound.

6.7 Safe Horizon

Definition 6.17 (Safe horizon). Given a degradation rate d and an acceptability threshold $c_{\min}$, the safe horizon $N^*(c, d)$ is the latest step at which the tacit state still has acceptable credence:

$$N^*(c, d) = \max\{n \in \mathbb{N} \mid c'(n) \geq c_{\min}\} = \left\lfloor \frac{c - c_{\min}}{d} \right\rfloor$$

Theorem 6.18 (Safe horizon characterization). *The safe horizon satisfies:*

$$c'(N^*) \geq c_{\min} \wedge c'(N^* + 1) < c_{\min}$$

[Lean: Frfp. Core. DynamicLayer. safe_horizon_characterization]

The safe horizon determines when the pipeline *must* invoke a TE or HFD step — i.e., when human evaluation is no longer optional.

6.8 Tacit Irreversibility

Theorem 6.19 (Tacit irreversibility). *Once a pipeline transitions from explicit to tacit space (via RB), this transition cannot be reversed:*

$$\nexists \text{ sequence of primitives from } T_0 \text{ to } E_0$$

[Lean: Frfp. Core. TacitDependence]

This is a restatement of Theorem 1.7 (No-Return Principle) in dynamic terms: once human evaluation is required, machine processing alone cannot restore tacit validity.

7 Collective and Governance Layer

7.1 Overview

This chapter extends FRFP from single-pipeline interactions to multi-agent settings. Three layers build on the kernel:

1. **Epistemic algebra** — knowledge operators and belief revision for individual agents.
2. **Institutional layer** — population structures, credence distributions, and multi-agent impossibility results.
3. **Governance layer** — derived governance predicates and the safe extension theorem.

7.2 Epistemic Algebra

Definition 7.1 (Epistemic state). An epistemic state for an agent is a record encoding the agent’s current knowledge about the pipeline’s tacit validity. It includes:

- A credence value $c \in [0, 1]$.
- A belief set over possible pipeline configurations.
- An update history (finite sequence of knowledge operations).

[Lean: Frfp.Core.EpistemicAlgebra.EpistemicState]

Definition 7.2 (Knowledge operator). A knowledge operator K maps an epistemic state to a proposition:

$$K : \text{EpistemicState} \rightarrow \text{Prop}$$

[Lean: Frfp.Core.EpistemicAlgebra.KnowledgeOp]

Definition 7.3 (Belief revision). A belief revision operation updates an epistemic state given new evidence:

$$\text{BeliefRevision} : \text{EpistemicState} \times \text{Evidence} \rightarrow \text{EpistemicState}$$

Axiom 7.4 (Belief revision idempotence). Revising with the same evidence twice is the same as revising once:

$$\text{BeliefRevision}(\text{BeliefRevision}(s, e), e) = \text{BeliefRevision}(s, e)$$

[Lean: Frfp.Core.EpistemicAlgebra.belief_revision_idempotent]

7.3 Institutional Layer

Definition 7.5 (Population). A population is a finite set of agents $\mathcal{A} = \{a_1, \dots, a_n\}$ each carrying an epistemic state. [Lean: Frfp.Core.InstitutionalLayer]

Definition 7.6 (Population credence). The population credence $\bar{c}$ is the weighted average of individual credences:

$$\bar{c} = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} c(a)$$

Theorem 7.7 (Population collapse bound). *Under continuous degradation without human evaluation, population credence reaches zero in finite time:*

$$\exists N, \mathcal{C}'(N) = 0$$

[Lean: Frfp.Core.CollectiveLayer.population_collapse_bound]

7.4 Consensus Impossibility

Theorem 7.8 (No grounding by consensus). *A consensus operation over AI agents cannot produce tacit grounding. Formally, no combination of AI-only operations yields a correctness judgment:*

$$\text{ConsensusAI}(\mathcal{A}) \not\Rightarrow \text{CorrectnessJudgment.correct}$$

[Lean: Frfp.Core.CollectiveLayer.consensus_no_grounding]

This theorem formalizes a key FRFP claim: voting, averaging, or ensemble aggregation among AI systems cannot substitute for human tacit judgment.

Note 7.9. The theorem says explicit-space consensus (no matter how sophisticated) does not produce a `correct` `CorrectnessJudgment`. The latter requires a HFD primitive, which is human-exclusive. The proof follows from the ETS theorem: no AI operation produces a morphism targeting T_0 .

7.5 Governance Framework

Definition 7.10 (Strong IL — global intent consistency).

$$\text{IL}_{\text{global}}(\mathcal{T}) \iff \forall a, b \in \mathcal{T}, a.\text{isAI} = \top \wedge b.\text{isAI} = \top \Rightarrow a.\text{intentTag} = b.\text{intentTag}$$

[Lean: Frfp.Minimal.BasisSweep.IL_global]

Definition 7.11 (Strong AR — full ambiguity resolution).

$$\text{AR}_{\text{global}}(\mathcal{T}) \iff \forall a \in \mathcal{T}, a.\text{ambiguous} = \top \Rightarrow a.\text{clarified} = \top$$

[Lean: Frfp.Minimal.BasisSweep.AR_global]

Definition 7.12 (Strong CSC — correctness lock propagation).

$$\text{CSC}_{\text{global}}(\mathcal{T}) \iff \left(\exists a \in \mathcal{T}, a.\text{locked} = \top \right) \Rightarrow \forall b \in \mathcal{T}, b.\text{locked} = \top$$

[Lean: Frfp.Minimal.BasisSweep.CSC_global]

Definition 7.13 (Strong governance admissibility).

$$\text{GA}(\mathcal{T}) \iff \text{IL}_{\text{global}}(\mathcal{T}) \wedge \text{AR}_{\text{global}}(\mathcal{T}) \wedge \text{CSC}_{\text{global}}(\mathcal{T})$$

[Lean: Frfp.Minimal.BasisSweep.GovernanceAdmissibleStrong]

7.6 Step Compatibility

Definition 7.14 (Step compatibility). A new step s is compatible with a strongly admissible trace $\mathcal{T}$ if:

1. **Intent compatibility:** $\forall a \in \mathcal{T}, a.\text{isAI} = \top \wedge s.\text{isAI} = \top \Rightarrow a.\text{intentTag} = s.\text{intentTag}$
2. **Ambiguity compatibility:** $s.\text{ambiguous} = \top \Rightarrow s.\text{clarified} = \top$
3. **Lock forward:** $(\exists a \in \mathcal{T}, a.\text{locked} = \top) \Rightarrow s.\text{locked} = \top$
4. **Lock backward:** $s.\text{locked} = \top \Rightarrow \forall a \in \mathcal{T}, a.\text{locked} = \top$

[Lean: Frfp.Minimal.BasisSweep.StepCompatibleStrong]

7.7 Governance Safe Extension Theorem

Theorem 7.15 (Governance safe extension). *Strong governance admissibility is preserved under compatible one-step extension:*

$$\text{GA}(\mathcal{T}) \wedge \text{Compatible}(\mathcal{T}, s) \Rightarrow \text{GA}(\mathcal{T} ++ [s])$$

[Lean: Frfp.Minimal.BasisSweep.governance_safe_extension_theorem] — machine-checked proof.

Proof. By case analysis on the membership of an arbitrary element of $\mathcal{T} ++ [s]$: it is either in $\mathcal{T}$ (where $\text{GA}(\mathcal{T})$ applies) or equals s (where the compatibility conditions apply). $\square \square$

This theorem is the key safety result for FRFP governance: governance compliance can be checked incrementally without re-auditing the entire trace history.

7.8 Entropy Drift in Collective Settings

Theorem 7.16 (Collective entropy drift). *Under AI-only rounds (no TE or HFD steps from any agent), the population-level entropy is non-decreasing:*

$$H_{\text{pop}}(t_2) \geq H_{\text{pop}}(t_1) \quad \text{for } t_1 \leq t_2 \text{ with no human steps}$$

[Lean: Frfp.Core.CollectiveLayer]

7.9 Policy Residue in the Collective Layer

Property	Mathematical skeleton	Policy residue
IL	Intent tag invariant over traces	Intent is a shared governance resource
AR	Every ambiguity flag implies clarification flag	Ambiguity is not acceptable in production
CSC	Correctness lock is trace-monotone	Once locked, correctness is never revisited
HEG/HEC	Tacit primitives target T_0	Only humans may ground these
Consensus impossibility	No AI primitive targets T_0	AI consensus cannot substitute for human judgment

8 Novel Results from Formalization

8.1 Overview

Three non-trivial theorems emerge directly from the Lean formalization — results not present in the original published theory and arising solely from the proof obligations imposed by the formal system. This chapter presents these theorems, their proofs, their interpretations, and the open questions they suggest.

Additionally, 8 direct consequence theorems from the reduced basis are catalogued.

8.2 Novel Theorem 1: First Unresolved Witness

Theorem 8.1 (First unresolved witness). *If an unresolved ambiguity exists anywhere in a protocol trace, there is a canonical first such occurrence:*

$$\begin{aligned} \forall \mathcal{T}, \left(\exists n, \mathcal{T}[n].\text{ambiguous} = \top \wedge \mathcal{T}[n].\text{clarified} = \perp \right) \Rightarrow \\ \exists n^*, \left(\mathcal{T}[n^*].\text{ambiguous} = \top \wedge \mathcal{T}[n^*].\text{clarified} = \perp \right) \wedge \\ \left(\forall m < n^*, \neg(\mathcal{T}[m].\text{ambiguous} = \top \wedge \mathcal{T}[m].\text{clarified} = \perp) \right) \end{aligned}$$

[Lean: Frfp.Minimal.BasisSweep.first_unresolved_witness] — machine-checked proof.

Proof. Let $P(n) = \text{"step } n \text{ is unresolved"}$. By hypothesis, $\exists n, P(n)$. By Lean's `Nat.find`, there exists a minimal witness $n^* = \text{Nat.find}(P)$ satisfying $P(n^*)$, and for all $m < n^*$, $\neg P(m)$. $\square$ $\square$

Why novel. The original AR axiom says "all ambiguities are resolved" — a global property. This theorem localizes AR violations: when AR fails, it fails at a specific first step. This is essential for:

- **Audit trails:** governance audits can find the exact first violation point.
- **Incremental checking:** a trace is AR-compliant up to step n iff $n < n^*$.
- **Repair protocols:** a minimal repair targets only step n^* , not the entire trace.

8.3 Novel Theorem 2: Governance Safe Extension

Theorem 8.2 (Governance safe extension). *Strong governance admissibility (IL, AR, CSC) is preserved under compatible one-step trace extension:*

$$\text{GA}(\mathcal{T}) \wedge \text{Compatible}(\mathcal{T}, s) \Rightarrow \text{GA}(\mathcal{T} \uparrow [s])$$

[Lean: Frfp.Minimal.BasisSweep.governance_safe_extension_theorem] — machine-checked proof.

Proof. Each of the three conjuncts of GA is preserved independently:

IL: For any two AI steps a, b in $\mathcal{T} ++ [s]$, if both are in $\mathcal{T}$ then $\text{IL}_{\text{global}}(\mathcal{T})$ applies; if one is s and the other is in $\mathcal{T}$, the intent compatibility condition applies.

AR: For any ambiguous step in $\mathcal{T} ++ [s]$: if in $\mathcal{T}$, $\text{AR}_{\text{global}}(\mathcal{T})$ applies; if it is s , the ambiguity compatibility condition applies.

CSC: The lock-forward and lock-backward conditions of compatibility ensure the lock propagates consistently. □

□

Why novel. The original FRFP governance axioms are stated as global properties of a complete trace. Theorem 7.2 is an *inductive* property — it says governance compliance is closed under one-step extension. This enables:

- **Online governance checking:** evaluate each new step independently.
- **Compositional reasoning:** compliant sub-traces compose to a compliant whole.
- **Distributed protocols:** each agent can enforce local step compatibility; global compliance follows.

8.4 Novel Theorem 3: Protocol Normal Form

Theorem 8.3 (Protocol normal form). *Every strongly admissible primitive trace decomposes into an explicit block, an optional boundary crossing, and a tacit block:*

$$\begin{aligned} \forall \mathcal{T} : \text{List}(\text{Primitive}), \text{AdmissibleStrong}(\mathcal{T}) \Rightarrow \\ \exists e_1 \dots e_k, h_1 \dots h_m, b \in \mathbb{B}, \\ \mathcal{T} = [e_1, \dots, e_k] ++ (\text{if } b \text{ then } [\text{RB}] \text{ else } []) ++ [h_1, \dots, h_m] \end{aligned}$$

where each $e_i \in \mathbf{E}_{\text{cat}} = \{\text{RI}, \text{EC}, \text{ED}\}$ and each $h_j \in \mathbf{T}_{\text{cat}} \setminus \{\text{RB}\} = \{\text{TE}, \text{HFD}\}$.

[Lean: *Frfp.Minimal.BasisSweep.protocol_normal_form_theorem*]—
machine-checked proof.

Why novel. The three-block structure is mentioned informally in the original theory. Theorem 7.3 makes it a formal theorem:

- **Every** admissible trace has this structure — it is not optional.
- The decomposition is canonical: the explicit block always precedes RB, which always precedes the tacit block.
- The boundary RB appears **at most once** (by Theorem 1.2).

Implication for protocol design. Any protocol that claims to be FRFP-compliant but allows RB in a non-terminal position, or that interleaves explicit and tacit steps, is not strongly admissible.

8.5 Direct Consequences of the Reduced Basis

Theorem 8.4 (Tacit source classification). $\forall p : \text{Primitive}, p.\text{source} = T_0 \Rightarrow p \in \{\text{TE}, \text{HFD}\}$ [*Lean: Frfp.Minimal.BasisConsequences.tacit_source_classification*]

Theorem 8.5 (Tacit source stays tacit). $\forall p, p.\text{source} = T_0 \Rightarrow p.\text{target} = T_0$ [*Lean: Frfp.Minimal.BasisConsequences.tacit_source_stays_tacit*]

Theorem 8.6 (AI steps never target tacit space). $\forall p, \text{is_AI_executable}(p) = \top \Rightarrow p.\text{target} \neq T_0$ [*Lean: Frfp.Minimal.BasisConsequences.ai_never_targets_tacit*]

Theorem 8.7 (AI/human disjoint). $\forall p, \neg(\text{is_AI_executable}(p) \wedge \text{is_human_only}(p))$ [*Lean: Frfp.Minimal.BasisConsequences.ai_human_disjoint*]

Theorem 8.8 (Unique boundary entry). $|\{p : \text{Primitive} \mid p.\text{source} = E_0 \wedge p.\text{target} = T_0\}| = 1$ [*Lean: Frfp.Minimal.BasisConsequences.unique_boundary_entry*]

Theorem 8.9 (IL pairwise intent consistency). *Under IL, any two AI steps in the same trace share an intent tag.* [*Lean: Frfp.Minimal.BasisConsequences.IL_pairwise_intent_consistency*]

Theorem 8.10 (AR: no unresolved ambiguity). *Under AR, no ambiguous step remains unclarified.* [*Lean: Frfp.Minimal.BasisConsequences.AR_no_unresolved_ambiguity*]

Theorem 8.11 (CSC: lock is monotone). *Under CSC, the correctness-locked flag is monotone non-decreasing over the trace.* [*Lean: Frfp.Minimal.BasisConsequences.CSC_lock_is_monotone*]

8.6 Novelty Classification

Theorem	Type	How it arose
7.1 First unresolved witness	New result	<code>Nat.find</code> forced minimal witness existence
7.2 Governance safe extension	New inductive property	Needed to prove GA is inductive
7.3 Protocol normal form	New completeness theorem	Admissibility definition crystallized three-block shape
Thms 7.4–7.11	Direct consequences	Short proofs from basis definitions

8.7 Open Questions

- Q1 Complete runToω proof.** The injectivity and round-trip properties are axiomatized. A full proof requires constructing a measurable bijection between the discrete run type and the measure-theoretic trajectory space.
- Q2 Phase 2 primitives.** The current kernel is Phase 1 (6 primitives). Can a Phase 2 extension be defined such that the analogues of ETS and RB uniqueness remain derivable theorems?
- Q3 Mechanized minimality.** The minimality of the 7-axiom basis is established by manual witness construction. An automated model-finder could confirm or refute this mechanically.

- Q4 Semantic grounding decidability.** The `CorrectnessJudgment` type has value unknown for undecidable cases. Is there a formal characterization of when grounding is decidable vs. undecidable?
- Q5 Probabilistic confluence.** Theorems 5.3–5.7 are deterministic properties of the hazard/survival functions. Is there a probabilistic confluence theorem: does the distribution over normal forms converge under repeated random reduction?